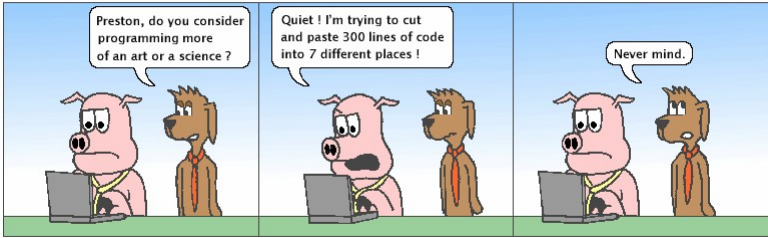


Hackles

By Drake Emko & Jen Brodzik



<http://hackles.org>

Copyright © 2001 Drake Emko & Jen Brodzik

Clean Functions

www.cs.uoi.gr/~zarras/soft-devII.htm

from Clean Code by R. C. Martin, a.k.a "Uncle Bob"

1

Function size and responsibilities

2

1

Can You Figure Out What it Does ?

Listing 3-1

HtmlUtil.java (FitNesse 20070619)

```

public static String testableHtml(
    PageData pageData,
    boolean includeSuiteSetup
) throws Exception {
    WikiPage wikiPage = pageData.getWikiPage();
    StringBuffer buffer = new StringBuffer();
    if (pageData.hasAttribute("Test")) {
        if (includeSuiteSetup) {
            WikiPage suiteSetup =
                PageCrawlerImpl.getInheritedPage(
                    SuiteResponder.SUITE_SETUP_NAME, wikiPage
                );
            if (suiteSetup != null) {
                WikiPagePath pagePath =
                    suiteSetup.getPageCrawler().getFullPath(suiteSetup);
                String pagePathName = PathParser.render(pagePath);
                buffer.append("include -setup .")
                    .append(pagePathName)
                    .append("\n");
            }
        }
        WikiPage setup =
            PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
        if (setup != null) {
            WikiPagePath setupPath =
                wikiPage.getPageCrawler().getFullPath(setup);
            String setupPathName = PathParser.render(setupPath);
            buffer.append("include -setup .")
                .append(setupPathName)
                .append("\n");
        }
    }
    buffer.append(pageData.getContent());
    if (pageData.hasAttribute("Test")) {
        WikiPage teardown =
            PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
        if (teardown != null) {
            WikiPagePath teardownPath =
                wikiPage.getPageCrawler().getFullPath(teardown);
            String teardownPathName = PathParser.render(teardownPath);
            buffer.append("\n")
                .append("include -teardown .")
                .append(teardownPathName)
                .append("\n");
        }
    }
}

```

Listing 3-1 (continued)

HtmlUtil.java (FitNesse 20070619)

```

if (includeSuiteSetup) {
    WikiPage suiteTeardown =
        PageCrawlerImpl.getInheritedPage(
            SuiteResponder.SUITE_TEARDOWN_NAME,
            wikiPage
        );
    if (suiteTeardown != null) {
        WikiPagePath pagePath =
            suiteTeardown.getPageCrawler().getFullPath(suiteTeardown);
        String pagePathName = PathParser.render(pagePath);
        buffer.append("include -teardown .")
            .append(pagePathName)
            .append("\n");
    }
}
pageData.setContent(buffer.toString());
return pageData.getHtml();
}

```

3

Can You Figure Out What it Does ?

Listing 3-1

HtmlUtil.java (FitNesse 20070619)

```

public static String testableHtml(
    PageData pageData,
    boolean includeSuiteSetup
) throws Exception {
    WikiPage wikiPage = pageData.getWikiPage();
    StringBuffer buffer = new StringBuffer();
    if (pageData.hasAttribute("Test")) {
        if (includeSuiteSetup) {
            WikiPage suiteSetup =
                PageCrawlerImpl.getInheritedPage(
                    SuiteResponder.SUITE_SETUP_NAME, wikiPage
                );
            if (suiteSetup != null) {
                WikiPagePath pagePath =
                    suiteSetup.getPageCrawler().getFullPath(suiteSetup);
                String pagePathName = PathParser.render(pagePath);
                buffer.append("include -setup .")
                    .append(pagePathName)
                    .append("\n");
            }
        }
        WikiPage setup =
            PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
        if (setup != null) {
            WikiPagePath setupPath =
                wikiPage.getPageCrawler().getFullPath(setup);
                String setupPathName = PathParser.render(setupPath);
                buffer.append("include -setup .")
                    .append(setupPathName)
                    .append("\n");
        }
    }
    buffer.append(pageData.getContent());
    if (pageData.hasAttribute("Test")) {
        WikiPage teardown =
            PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
        if (teardown != null) {
            WikiPagePath teardownPath =
                wikiPage.getPageCrawler().getFullPath(teardown);
                String teardownPathName = PathParser.render(teardownPath);
                buffer.append("\n")
                    .append("include -teardown .")
                    .append(teardownPathName)
                    .append("\n");
        }
    }
}

```

Listing 3-1 (continued)

HtmlUtil.java (FitNesse 20070619)

```

if (includeSuiteSetup) {
    WikiPage suiteTeardown =
        PageCrawlerImpl.getInheritedPage(
            SuiteResponder.SUITE_TEARDOWN_NAME,
            wikiPage
        );
    if (suiteTeardown != null) {
        WikiPagePath pagePath =
            suiteTeardown.getPageCrawler().getFullPath(suiteTeardown);
        String pagePathName = PathParser.render(pagePath);
        buffer.append("include -teardown .")
            .append(pagePathName)
            .append("\n");
    }
}
pageData.setContent(buffer.toString());
return pageData.getHtml();
}

```

Probably not !!

Not only is it **long**, but it's got **duplicated code**, lots of **odd strings**, and many **strange data types and APIs**.

4

2

Small !!

Listing 3-3

HtmlUtil.java (re-refactored)

```
public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}
```

the first rule is that **functions should be small !!!**

5

Small !!

Listing 3-3

HtmlUtil.java (re-refactored)

```
public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}
```

What does small mean ???

hundreds of lines ??

tens of lines ??

less ??

6

3

Small !!

Listing 3-3

HtmlUtil.java (re-refactored)

```
public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}
```

In the **eighties** they used to say that a function should be **no bigger** than a **screen-full**.

At that time **VT100 screens** were **24 lines by 80 columns**, and the editors used 4 lines for administrative purposes.

Nowadays with a cranked-down font and a nice big monitor, you can fit 150 characters on a line and a **100 lines** or more on a **screen**.

Lines should not be 150 characters long. Functions should not be 100 lines long.

Functions should hardly ever be 20 lines long !!!!

7

Small !!



... we can make large programs with *two, or three, or four lines* functions

e.g. Kent Beck's Sparkle graphical application...

practically → it is quite reasonable to make algorithms of 2, 3, 4 steps, consisting of more detailed algorithms of 2, 3, 4 steps, etc.

8

Small !!

Listing 3-3

HtmlUtil.java (re-refactored)

```
public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}
```

What does it take to make so small functions ???

Blocks within **if block statements**, **else block statements**, **while block statements**, and so on should be **one line long**.

Probably that line should be a function call.

Not only does this keep the enclosing function small, but it also **adds documentary value** because the function called within the block can have a **nicely descriptive name**.

9

Small !!

Listing 3-3

HtmlUtil.java (re-refactored)

```
public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}
```

Functions should not be large enough to hold nested blocks.

Therefore, **the indent level** of a function should not be greater than **one** or **two**.

This, of course, makes the functions easier to read and understand.

10

5

Do One Thing !!

The following advice has appeared in one form or another for 30 years or more.



**FUNCTIONS SHOULD DO ONE THING.
THEY SHOULD DO IT WELL.
THEY SHOULD DO IT ONLY.**

11

Do One Thing !!

The following advice has appeared in one form or another for 30 years or more.



**FUNCTIONS SHOULD DO ONE THING.
THEY SHOULD DO IT WELL.
THEY SHOULD DO IT ONLY.**

Great but the problem is that it is hard to know **what one thing is !!!**

12

Do One Thing !!

Listing 3-3

HtmlUtil.java (re-refactored)

```
public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}
```

Does this code do one thing?

13

Do One Thing !!

Listing 3-3

HtmlUtil.java (re-refactored)

```
public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}
```

Does this code do one thing?

It's easy to make the case that it's doing **three things**:

1. Check whether pageData refer to a test page.
2. If so, includes setup and teardown data to pageData.
3. Renders pageData in HTML.

14

Do One Thing !!

Listing 3-3

HtmlUtil.java (re-refactored)

```
public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}
```

1. Check whether pageData refer to a test page.
2. If so, includes setup and teardown parts to pageData.
3. Renders pageData in HTML.

These three steps are at the **same level of abstraction** (detail) **one level below** the stated **name of the function**; they explain how to construct a page that comprises setup and teardown parts.

15

Do One Thing !!

To describe what the function does we need only **a simple TO paragraph (1,2 simple sentences)....**

TO RenderPageWithSetupsAndTeardowns, we check to see whether the pageData refers to a test page and if so, we include the setup and teardown parts. We conclude by extracting the HTML code that contains the pageData.

16

Do One Thing !!

Listing 3-1
HtmlUtil.java (FitNesse 20070619)

```
public static String testableHtml(
    PageData pageData,
    boolean includeSuiteSetup
) throws Exception {
    WikiPage wikiPage = pageData.getWikiPage();
    StringBuffer buffer = new StringBuffer();
    if (pageData.hasAttribute("Test")) {
        if (includeSuiteSetup) {
            WikiPage suiteSetup =
                PageCrawlerImpl.getInheritedPage(
                    SuiteResponder.SUITE_SETUP_NAME, wikiPage
                );
            if (suiteSetup != null) {
                WikiPagePath pagePath =
                    suiteSetup.getPageCrawler().getFullPath(suiteSetup);
                String pagePathName = PathParser.render(pagePath);
                buffer.append("!!include -setup ." +
                    pagePathName +
                    ".append(\"\\n\");");
            }
        }
        WikiPage setup =
            PageCrawlerImpl.getInheritedPage(
                SuiteResponder.SUITE_SETUP_NAME, wikiPage
            );
        if (setup != null) {
            WikiPagePath setupPath =
                setup.getPageCrawler().getFullPath(setup);
            String setupPathName = PathParser.render(setupPath);
            buffer.append("!!include -setup ." +
                setupPathName +
                ".append(\"\\n\");");
        }
    }
    buffer.append(pageData.getHtml());
    if (pageData.hasAttribute("Test")) {
        WikiPage teardown =
            PageCrawlerImpl.getInheritedPage(
                SuiteResponder.SUITE_TEARDOWN_NAME, wikiPage
            );
        if (teardown != null) {
            WikiPagePath teardownPath =
                teardown.getPageCrawler().getFullPath(teardown);
            String teardownPathName = PathParser.render(teardownPath);
            buffer.append("!!include -teardown ." +
                teardownPathName +
                ".append(\"\\n\");");
        }
    }
}
```

Listing 3-1 (continued)
HtmlUtil.java (FitNesse 20070619)

```
if (includeSuiteSetup) {
    WikiPage suiteTeardown =
        PageCrawlerImpl.getInheritedPage(
            SuiteResponder.SUITE_TEARDOWN_NAME,
            wikiPage
        );
    if (suiteTeardown != null) {
        WikiPagePath pagePath =
            suiteTeardown.getPageCrawler().getFullPath(suiteTeardown);
        String pagePathName = PathParser.render(pagePath);
        buffer.append("!!include -teardown ." +
            pagePathName +
            ".append(\"\\n\");");
    }
}
pageData.setContent(buffer.toString());
return pageData.getHtml();
}
```

To explain what this function does we would need **many nested TO paragraphs**

almost every nested block of commands is by itself a TO paragraph explaining how to realize a step of the higher level block

This indicates that it does more than one thing

17

Step down rule

Listing 3-3

HtmlUtil.java (re-refactored)

```
public static String renderPageWithSetupsAndTeardowns(
    PageData pageData, boolean isSuite) throws Exception {
    if (isTestPage(pageData))
        includeSetupAndTeardownPages(pageData, isSuite);
    return pageData.getHtml();
}
```

We want the code to read like a **top-down narrative**.

We want **every function** to be **followed** by the functions that it calls, so that we can read the program, descending one level of abstraction (detail) at a time.

To say this differently, we want to be able to read the program as though it were a set of **TO paragraphs**, each of which is describing the current level of abstraction and **referencing subsequent TO paragraphs** at the next level down.

18

Switch Statements & if/else chains

```
class Bird{
    // ...
    private int type;
    private static final int EUROPEAN = 0;
    private static final int AFRICAN = 1;
    private static final int NORVEGIAN_BLUE = 2;
    public double getSpeed(){
        switch(type) {
            case EUROPEAN: return getBaseSpeed();
            case AFRICAN: return getBaseSpeed() - getLoadFactor()*numberOfCoconuts;
            case NORVEGIAN_BLUE: return (isNailed) ? 0 : getBaseSpeed();
        }
    }
    // ...
}
```

It's hard to make a **small switch** statement.

By their nature, switch statements always **do N things**.

19

Switch Statements & if/else chains

```
class Bird{
    // ...
    private int type;
    private static final int EUROPEAN = 0;
    private static final int AFRICAN = 1;
    private static final int NORVEGIAN_BLUE = 2;
    public double getSpeed(){
        switch(type) {
            case EUROPEAN: return getBaseSpeed();
            case AFRICAN: return getBaseSpeed() - getLoadFactor()*numberOfCoconuts;
            case NORVEGIAN_BLUE: return (isNailed) ? 0 : getBaseSpeed();
        }
    }
    // ...
}
```

The worst thing is that they never appear once !!

Why is that ???

20

10

Switch Statements & if/else chains

```
class Bird{
    // ...
    private int type;
    private static final int EUROPEAN = 0;
    private static final int AFRICAN = 1;
    private static final int NORVEGIAN_BLUE = 2;
    public double getSpeed(){
        switch(type) {
            case EUROPEAN: return getBaseSpeed();
            case AFRICAN: return getBaseSpeed() - getLoadFactor()*numberOfCoconuts;
            case NORVEGIAN_BLUE: return (isNailed) ? 0 : getBaseSpeed();
        }
    }
    // ...
}
```

The worst thing is that they never appear once !!

Most likely there are:

- more Bird methods like this one,
- several places where Bird objects are created... in these places the code has a similar structure that chooses what kind of Bird object to create !!!

21

Switch Statements & if/else chains

<pre>// somewhere else ... Bird newBird; if(birdSelectionFromGui.equals("European")){ newBird = new Bird(0, ""); } else if(birdSelectionFromGui.equals("African", "")){ newBird = new Bird(1, ""); } else if(birdSelectionFromGui.equals("Norwegian", "")){ newBird = new Bird(2, ""); } }</pre>	<pre>// somewhere else ... Bird birdFromFile; if(birdSelectionFromFile.equals("European")){ birdFromFile = new Bird(0, fileName); } else if(birdSelectionFromFile.equals("African")){ birdFromFile = new Bird(1, fileName); } else if(birdSelectionFromFile.equals("Norwegian")){ birdFromFile = new Bird(2, fileName); } }</pre>
--	---

The worst thing is that they never appear once !!

Most likely there are:

- more Bird methods like this one,
- several places where Bird objects are created... in these places the code has a similar structure that chooses what kind of Bird object to create !!!

22

11

Switch Statements & if/else chains

```
class Bird{
    // ...
    private int type;
    private static final int EUROPEAN = 0;
    private static final int AFRICAN = 1;
    private static final int NORVEGIAN_BLUE = 0;
    public double getSpeed(){
        switch(type) {
            case EUROPEAN: return getBaseSpeed();
            case AFRICAN: return getBaseSpeed() - getLoadFactor()*numberOfCoconuts;
            case NORVEGIAN_BLUE: return (isNailed) ? 0 : getBaseSpeed();
        }
    }
    // ...
}
```

Unfortunately **we can't always** avoid switch statements.

Here we can, but how ??

23

Switch Statements & if/else chains

```
class Bird{
    // ...
    private int type;
    private static final int EUROPEAN = 0;
    private static final int AFRICAN = 1;
    private static final int NORVEGIAN_BLUE = 0;
    public double getSpeed(){
        switch(type) {
            case EUROPEAN: return getBaseSpeed();
            case AFRICAN: return getBaseSpeed() - getLoadFactor()*numberOfCoconuts;
            case NORVEGIAN_BLUE: return (isNailed) ? 0 : getBaseSpeed();
        }
    }
    // ...
}
```

In many cases we can replace switch statements with polymorphism !!!

24

12

Switch Statements & if/else chains

```
abstract class Bird{
    public abstract double getSpeed();
    // ...
}
class European extends Bird {
    public double getSpeed(){return getBaseSpeed();}
}
class African extends Bird {
    public double getSpeed(){
        return getBaseSpeed() - getLoadFactor()*_numberOfCoconuts;
    }
}
.....
```

In many cases we can replace switch statements with polymorphism !!!

Did we solve the problem ??

25

Switch Statements & if/else chains

```
abstract class Bird{
    public abstract double getSpeed();
    // ...
}
class European extends Bird {
    public double getSpeed(){return getBaseSpeed();}
}
class African extends Bird {
    public double getSpeed(){
        return getBaseSpeed() - getLoadFactor()*_numberOfCoconuts;
    }
}
.....
```

Did we solve the problem ??

We did for class Bird,

but, how about the rest of the places where **Bird** objects are created... in these places **the code has a switch structure** that **chooses** which **subclass of Bird** objects to create !!!

26

13

Switch Statements & if/else chains

```
// somewhere else ...
Bird newBird;
if(birdSelectionFromGui.equals("European")){
    newBird = new European("");
} else
    if(birdSelectionFromGui.equals("African")){
        newBird = new African("");
    } else
        if(birdSelectionFromGui.equals("Norwegian")){
            newBird = new Norwegian("");
        }
        }

// somewhere else ...
Bird birdFromFile;
if(birdSelectionFromFile.equals("European")){
    birdFromFile = new European(fileName);
} else
    if(birdSelectionFromFile.equals("African")){
        birdFromFile = new African(fileName);
    } else
        if(birdSelectionFromFile.equals("Norwegian")){
            birdFromFile = new Norwegian(fileName);
        }
        }
```

But, how about the rest of the places where Bird objects are created... in these places the code has a switch structure that chooses which subclass of Bird objects to create !!!

What can we do to fix this ???

27

Switch Statements & if/else chains

```
// somewhere else ...
Bird newBird;
if(birdSelectionFromGui.equals("European")){
    newBird = new European("");
} else
    if(birdSelectionFromGui.equals("African")){
        newBird = new African("");
    } else
        if(birdSelectionFromGui.equals("Norwegian")){
            newBird = new Norwegian("");
        }
        }

// somewhere else ...
Bird birdFromFile;
if(birdSelectionFromFile.equals("European")){
    birdFromFile = new European(fileName);
} else
    if(birdSelectionFromFile.equals("African")){
        birdFromFile = new African(fileName);
    } else
        if(birdSelectionFromFile.equals("Norwegian")){
            birdFromFile = new Norwegian(fileName);
        }
        }
```

What can we do to fix this ???

The best we can do is to keep the object creation logic **buried in one place**, and reuse it in all the different places that need to create (a subclass of) Bird objects

This place is often called a **Factory class**

28

14

Switch Statements & if/else chains

```
class BirdFactory{
    private final int EUROPEAN = 0;
    private final int AFRICAN = 1;
    private final int NORVEGIAN_BLUE = 0;
    public Bird createBird(String type, String fileName){
        if(type.equals("European"))
            return new European(fileName);
        if(type.equals("African"))
            return new African(fileName);
        if(type.equals("Norwegian"))
            return new Norwegian(fileName);
    }
    // ...
}
```

This place is often called a **factory class**

What is the **benefit** of that ??

29

Switch Statements & if/else chains

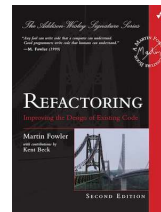
```
class BirdFactory{
    private static final int EUROPEAN = 0;
    private static final int AFRICAN = 1;
    private static final int NORVEGIAN_BLUE = 0;
    public static Bird createBird(String type, String fileName){
        if(type.equals("European"))
            return new European(fileName);
        if(type.equals("African"))
            return new African(fileName);
        if(type.equals("Norwegian"))
            return new Norwegian(fileName);
    }
    // ...
}
```

This place is often called a **factory class**

What is the **benefit** of that ??

Additions/removals/changes in **one place**

30



Smells and related refactorings

<https://refactoring.com/catalog/>

31

Long Method

Symptoms

Large number of lines in the method. (immediately suspicious of any method with more than **five to ten lines**.)

Causes

A method has started down a path, and rather than break the flow or identify the helper objects, the author **adds "one more thing."**

Code is often **easier to write than it is to read**, so there's a temptation to write blocks that are too big.

32

16

Long Method

What to Do

Use **Extract Method** to break up the method into smaller pieces. Look for comments or white space that delineate “interesting” blocks.

33

Long Method

What to Do

You may find **other related refactorings** (those that clean up straight-line code, **conditionals**, and **variable usage**) helpful before you even begin splitting up the method.

34

Inline Temp

You have a temp that is assigned to once with a simple expression, and the temp is getting in the way of other refactorings.

Replace all references to that temp with the expression.

```
double basePrice = anOrder.basePrice();
return (basePrice > 1000)
```



```
return (anOrder.basePrice() > 1000)
```

35

Replace Temp with Query

You are using a temporary variable to hold the result of an expression.

Extract the expression into a method. Replace all references to the temp with the expression. The new method can then be used in other methods.

```
double basePrice = _quantity * _itemPrice;
if (basePrice > 1000)
    return basePrice * 0.95;
else
    return basePrice * 0.98;
```



```
if (basePrice() > 1000)
    return basePrice() * 0.95;
else
    return basePrice() * 0.98;
...
double basePrice() {
    return _quantity * _itemPrice;
}
```

36

zas1

Replace Temp with Query

Replace Temp with Query often is a **vital step** before *Extract Method*.

Local variables make it **difficult to extract**, so **replace as many variables** as you can with **queries**.

The straightforward cases of this refactoring are those in which temps are **assigned only to once**. Other cases are trickier (e.g. see Split Temporary Variable) but possible.

37

Split Temporary Variable

You have a temporary variable assigned to more than once, but is not a loop variable nor a collecting temporary variable.

Make a separate temporary variable for each assignment.

```
double temp = 2 * (_height + _width);
System.out.println (temp);
temp = _height * _width;
System.out.println (temp);
```



```
final double perimeter = 2 * (_height + _width);
System.out.println (perimeter);
final double area = _height * _width;
System.out.println (area);
```

38

19

Διαφάνεια 37

zas1 why is this an issue ??
zarras; 17/5/2012

Split Temporary Variable

Temporary variables are made for various uses.

Some of these uses naturally lead to the temp's being assigned to several times. **Loop variables** change for each run around a loop (such as the `i` in `for (int i=0; i<10; i++)`). Collecting temporary variables collect together some value that is built up during the method.

Many **other temporaries** are used to hold the result of a bit of code for easy reference later.

These kinds of variables **should be set only once**. That they are set more than once is a **sign that they have more than one responsibility** within the method.

Any variable with **more than one responsibility** should be replaced with a **temp for each responsibility**. Using a temp for two different things is **very confusing** for the reader.

39

Decompose Conditional

You have a complicated conditional (if-then-else) statement.

Extract methods from the condition, then part, and else parts.

```
if (date.before (SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * _winterRate + _winterServiceCharge;
else charge = quantity * _summerRate;
```



```
if (notSummer(date))
    charge = winterCharge(quantity);
else charge = summerCharge (quantity);
```

40

20

Decompose Conditional

One of the most **common areas of complexity** in a program lies in **complex conditional logic**. As you write code to test conditions and to do various things depending on various conditions, you quickly end up with a pretty **long method**. **Length** of a method is in itself a factor that makes it **harder to read**, but **conditions increase the difficulty**.

As with any large block of code, you can make your intention clearer by **decomposing** it and replacing chunks of code with a method call named after the intention of that block of code.

With conditions you can receive further benefit by doing this for the conditional part and each of the alternatives.

This way you highlight the condition and make it clearly what you are branching on. You also highlight the reason for the branching.

41

Consolidate Conditional

You have a sequence of conditional tests with the same result.

Combine them into a single conditional expression and extract it.

```
double disabilityAmount() {
    if (_seniority < 2) return 0;
    if (_monthsDisabled > 12) return 0;
    if (_isPartTime) return 0;
    // compute the disability amount
}
```



```
double disabilityAmount() {
    if (isNotEligibleForDisability()) return 0;
    // compute the disability amount
}
```

42

21

Consolidate Conditional

Sometimes you see a **series of conditional checks** in which each check is different yet the **resulting action** is the **same**. When you see this, you should use **ands** and **ors** to consolidate them into a **single conditional check** with a single result.

Consolidating the conditional code is important for two reasons.

First, it makes the **check clearer** by showing that you are really making a single check. The sequence has the same effect, but it communicates carrying out a sequence of separate checks that just happen to be done together.

The second reason for this refactoring is that it often sets you up for **EXTRACT METHOD** and **DECOMPOSE CONDITIONAL**.

The reasons in favor of consolidating conditionals also point to reasons for not doing it. **If you think the checks are really independent and shouldn't be thought of as a single check, don't do the refactoring.**

****** Check that any of the conditionals has side effects. If there are side effects, you won't be able to do this refactoring**

43

Consolidate Duplicate Conditional Fragments

The same fragment of code is in all branches of a conditional expression.

Move it outside of the expression.

```
if (isSpecialDeal()) {
    total = price * 0.95;
    send();
}
else {
    total = price * 0.98;
    send();
}
```



```
if (isSpecialDeal())
    total = price * 0.95;
else
    total = price * 0.98;
send();
```

44

22

Consolidate Duplicate Conditional Fragments

Sometimes you find **common code** executed in all **legs of a conditional**. In that case you should **move** the code to **outside** the conditional. This makes clearer what varies and what stays the same. Simplifies **DECOMPOSE CONDITIONAL** too.

If the common code is at the **beginning**, move it to **before the conditional**.

If the common code is at the **end**, move it to **after the conditional**.

If the common code is in the **middle**, **look to see whether the code before or after it changes anything**. Depending on this, you can move the common code forward or backward to the ends.

If there is **more than a single statement**, you should **extract** that code into **a method**.

45

Replace Nested Conditional with Guard

A method has conditional behavior that does not make clear what the normal path of execution is

Use Guard Clauses for all the special cases

```
double getPayAmount() {
    double result;
    if (!_isDead) result = deadAmount();
    else {
        if (!_isSeparated) result = separatedAmount();
        else {
            if (!_isRetired) result = retiredAmount();
            else result = normalPayAmount();
        }
    }
    return result;
};
```



```
double getPayAmount() {
    if (!_isDead) return deadAmount();
    if (!_isSeparated) return separatedAmount();
    if (!_isRetired) return retiredAmount();
    return normalPayAmount();
};
```

46

23

Replace Nested Conditional with Guard

We find that **conditional expressions** come in **two forms**.

- The **first form** is a check whether **either course** is part of the **normal behavior**.
- The **second form** is a situation in which **one branch** from the conditional indicates **normal behavior** and **the other** indicates an **unusual condition**.

These kinds of conditionals have different intentions, and these intentions should come through in the code.

- **If both are part of normal behavior**, use a condition with an **if and an else leg**.
- If the condition is an **unusual condition**, **check the condition** and **return** if the condition is **true**. This is called **guard clause**.

The key point about *Replace Nested Conditional with Guard Clauses* is one of **emphasis**. If you are using an **if-then-else** construct you are giving **equal weight** to the if leg and the else leg. This communicates to the reader that the legs are equally likely and important. Instead the **guard clause** says, "This is rare, and if it happens, do something and get out."

47

Function parameters

48

24

Function Arguments



What is the **ideal # of arguments** for a clean function ?

49

Function Arguments



What is the **ideal # of arguments** for a clean function ?

zero !! Niladic functions are the best

one (monadic functions), or **two** arguments (dyadic functions),
are also **OK**

Avoid functions with **3**, or **more** than 3 arguments !!!

Why many arguments are not good ??

50

25

Function Arguments

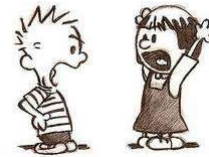
Arguments require a lot of brain cycles !!

Each time you use the function, have to remember the **arguments**, their **intent**, their **order**, etc.

Arguments are even **harder** from a **testing** point of view.

Imagine the difficulty of writing all the **test cases** to ensure that various **combinations of arguments** work properly.

With more than two arguments, testing every combination of appropriate values can become very painful



51

Common Monadic Forms

There are two very common reasons to pass a single argument.

You may be **asking a question** about that argument, as in:

```
boolean fileExists("MyFile")
```

Or you may be **operating** on that argument, **transforming** it into something else and **returning** it. For example,:

```
InputStream openFile("MyFile")
```

transforms a file name String into an InputStream return value.

Another very useful form for a single argument function, is an **event/command**. In this form there is an **input argument** but **no output argument**.

The function uses the argument to alter the state of the program, for example:

```
void parseStream(InputStream stream)
```



52

Separate Commands from Queries

```
public int parseFile(String fileName);  
.....
```

what is the **meaning** of this ?

Unclear what is returned by the function



53

Separate Commands from Queries

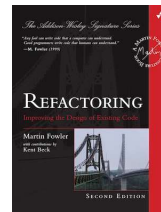
```
public void parseFile(String fileName);  
public int getNumberOfDataEntries();  
.....
```

Much better if we separate command from query



54

27



Smells and related refactorings

<https://refactoring.com/catalog/>

55

Long Parameter List

Symptoms

Count the number of parameters to a method.
(Even three or four might be too many.)

Causes

An author often tries to **minimize coupling between objects**. Instead of the called object being aware of relationships between objects, you let the caller locate everything; then the method concentrates on what it is being asked to do with the pieces.

Or, the author **generalizes** the routine to deal with **multiple variations**: there's a general algorithm and a lot of **"control" parameters**.

56

28

Long Parameter List

What to Do ^{z1}

For **control/flag parameters** *Replace Parameter with Explicit Methods*

57

Replace Parameter with Explicit Methods

You have a method that runs different code depending on the values of an enumerated parameter.

Create a separate method for each value of the parameter.

```
void setValue (String name, int value) {
    if (name.equals("height")) {
        _height = value;
        return;
    }
    if (name.equals("width")) {
        _width = value;
        return;
    }
    Assert.shouldNeverReachHere();
}
```



```
void setHeight(int arg) {
    _height = arg;
}
void setWidth (int arg) {
    _width = arg;
}
```

58

29

Διαφάνεια 57

z1 when you reach the what to do slides go back and forth to explain each bullet

this way it rolls better!!!

zarras; 30/11/2018

Replace Parameter with Explicit Methods

The usual case for this refactoring is that you have **discrete values** of a **parameter**, **test** for those values in a **conditional**, and do **different things**.

Remember, **methods** should do **one thing** that can be expressed in a simple **TO statement**.

Otherwise, the caller has to **decide** what it wants to do, **set the parameter**, and generally do **more work** than needed.

59

Long Parameter List

What to Do

If the **parameter** value can be **obtained from an object or method** that the called method already knows, ***Replace Parameter with Method***.

60

z2 when you reach the what to do slides go back and forth to explain each bullet

this way it rolls better!!!

zarras; 30/11/2018

Replace Parameter with Method

An object invokes a method, then passes the result as a parameter for a method. The receiver can also invoke this method.

Remove the parameter and let the receiver invoke the method.

```
int basePrice = _quantity * _itemPrice;
discountLevel = getDiscountLevel();
double finalPrice = discountedPrice (basePrice, discountLevel);
```



```
int basePrice = _quantity * _itemPrice;
double finalPrice = discountedPrice (basePrice);
```

61

Replace Parameter with Method

If a method can **get a value** that is passed in as **parameter** by **another means**, it should. **Long parameter lists** are difficult to understand, and we should reduce them as much as possible.

One way of **reducing parameter lists is to look to see whether the receiving method can make the same calculation.**

62

31

Long Parameter List

What to Do z3

If the parameters come from a **single object**, try **Preserve Whole Object**.

63

Preserve Object

You are getting several values from an object and passing these values as parameters in a method call.

Send the whole object instead.

```
int low = daysTempRange().getLow();  
int high = daysTempRange().getHigh();  
withinPlan = plan.withinRange(low, high);
```



```
withinPlan = plan.withinRange(daysTempRange());
```

64

32

z3 when you reach the what to do slides go back and forth to explain each bullet

this way it rolls better!!!

zarras; 30/11/2018

Preserve Object

Preserve Whole Object often makes the code more readable.

Long parameter lists can be hard to work with because both caller and callee have to remember which values were there.

There is a **down side**. When you pass in values, the called object has a **dependency** on the **values**, but there isn't any **dependency** to the **object** from which the values were extracted.

Passing in the required object causes a dependency between the required object and the called object. If this is going to mess up your dependency structure, **don't use *Preserve Whole Object***.

65

Long Parameter List

What to Do z4

If the data is not from one logical object, you still might **group them** via ***Introduce Parameter Object***.

66

z4 when you reach the what to do slides go back and forth to explain each bullet

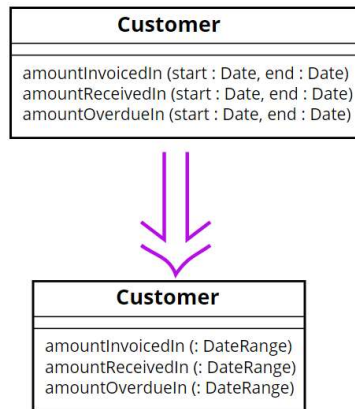
this way it rolls better!!!

zarras; 30/11/2018

Introduce Parameter Object

You have a group of parameters that naturally go together.

Replace them with an object.



67

Introduce Parameter Object

Often you see a particular **group of parameters** that tend to be **passed together**. Several methods may use this group, either on one class or in several classes.

Such a group of classes is a data clump and can be **replaced with an object** that carries all of this data. It is worthwhile to turn these parameters into objects just to group the data together.

This refactoring is useful because **it reduces the size of the parameter lists, and long parameter lists are hard to understand**.

You get a **deeper benefit**, however, because once you have clumped together the parameters, **you soon see behavior that you can also move into the new class**.

Often the bodies of the methods have common manipulations of the parameter values. By moving this behavior into the new object, you can remove a lot of duplicated code.

68

Error handling

69

What is wrong with error handling ?

```
if (deletePage(page) == E_OK) {  
    if (registry.deleteReference(page.name) == E_OK) {  
        if (configKeys.deleteKey(page.name.makeKey()) == E_OK) {  
            logger.log("page deleted");  
        } else {  
            logger.log("configKey not deleted");  
        }  
    } else {  
        logger.log("deleteReference from registry failed");  
    }  
} else {  
    logger.log("delete failed");  
    return E_ERROR;  
}
```



70

35

Prefer Exceptions to Error Codes

```
if (deletePage(page) == E_OK) {
    if (registry.deleteReference(page.name) == E_OK) {
        if (configKeys.deleteKey(page.name.makeKey()) == E_OK) {
            logger.log("page deleted");
        } else {
            logger.log("configKey not deleted");
        }
    } else {
        logger.log("deleteReference from registry failed");
    }
} else {
    logger.log("delete failed");
    return E_ERROR;
}
```



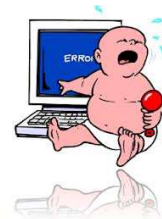
Returning **error codes** from command functions is a **subtle violation** of **command query separation**.

The practical problem is that the code that calls the function gets **complicated**, mixing **error handling** with **normal execution**.

71

Prefer Exceptions to Error Codes

```
try {
    deletePage(page);
    registry.deleteReference(page.name);
    configKeys.deleteKey(page.name.makeKey());
}
catch (Exception e) {
    logger.log(e.getMessage());
}
```



Here, each method throws an **exception**, instead of returned **error codes**.

Then the error processing code can be separated from the normal code using **try/catch**

The code is much **simplified !!**

72

Prefer Exceptions to Error Codes

```
try {
    deletePage(page);
    registry.deleteReference(page.name);
    configKeys.deleteKey(page.name.makeKey());
}
catch (Exception e) {
    logger.log(e.getMessage());
}
```



try/catch blocks are also considered ugly and hard to read...

it forces the reader to understand 2 things, how is the normal work done and how exceptions are handled !!

Can we do better ???

73

Prefer Exceptions to Error Codes

```
public void delete(Page page) {
    try {
        deletePageAndAllReferences(page);
    }
    catch (Exception e) {
        logError(e);
    }
}

private void deletePageAndAllReferences(Page page) throws Exception {
    deletePage(page);
    registry.deleteReference(page.name);
    configKeys.deleteKey(page.name.makeKey());
}

private void logError(Exception e) {
    logger.log(e.getMessage());
}
```



The above, provides a **better separation** between normal and error handling code that makes the code easier to understand and modify.

74

37

Provide Context with Exceptions

Each exception that you throw should provide enough context to determine the source and location of an error.

In Java, you can get a **stack trace** from any exception; however, a stack trace can't tell you the **intent of the operation that failed**.

Create **informative error messages** and pass them along with your exceptions.

Mention the **operation** that failed and the **type** of failure. If you are logging in your application, pass along **enough information** to be able to **log** the error in your catch.

75

Define Meaningful Exceptions wrt Caller's Needs

How about this code ??

```
ACMEPort port = new ACMEPort(12);

try {
    port.open();
} catch (DeviceResponseException e) {
    reportPortError(e);
    logger.log("Device response exception", e);
} catch (ATM1212UnlockedException e) {
    reportPortError(e);
    logger.log("Unlock exception", e);
} catch (GMXError e) {
    reportPortError(e);
    logger.log("Device response exception");
} finally {
    ...
}
```

76

38

Define Meaningful Exceptions wrt Caller's Needs

```
ACMEPort port = new ACMEPort(12);

try {
    port.open();
} catch (DeviceResponseException e) {
    reportPortError(e);
    logger.log("Device response exception", e);
} catch (ATM1212UnlockedException e) {
    reportPortError(e);
    logger.log("Unlock exception", e);
} catch (GMXError e) {
    reportPortError(e);
    logger.log("Device response exception");
} finally {
    ...
}
```

How about this code ??

A lot of exceptions for just one method call with weird names that don't make much sense for the caller of port.open();

The third party API that is used here provides a poor set of exception classes that obscure the readability of the code...

77

Define Meaningful Exceptions wrt Caller's Needs

when we define exception classes in an application, our most important concern should be to be meaningful for the code that catches and handles them.

78

Define Meaningful Exceptions wrt Caller's Needs

```
ACMEPort port = new ACMEPort(12);

try {
    port.open();
} catch (DeviceResponseException e) {
    reportPortError(e);
    logger.log("Device response exception", e);
} catch (ATM1212UnlockedException e) {
    reportPortError(e);
    logger.log("Unlock exception", e);
} catch (GMXError e) {
    reportPortError(e);
    logger.log("Device response exception");
} finally {
    ...
}
```

How can we deal with poor exceptions that come from an external API ??

79

Define Meaningful Exceptions

```
LocalPort port = new LocalPort(12);
try {
    port.open();
} catch (PortDeviceFailure e) {
    reportError(e);
    logger.log(e.getMessage(), e);
} finally {
    ...
}

public class LocalPort {
    private ACMEPort innerPort;

    public LocalPort(int portNumber) {
        innerPort = new ACMEPort(portNumber);
    }

    public void open() {
        try {
            innerPort.open();
        } catch (DeviceResponseException e) {
            throw new PortDeviceFailure(e);
        } catch (ATM1212UnlockedException e) {
            throw new PortDeviceFailure(e);
        } catch (GMXError e) {
            throw new PortDeviceFailure(e);
        }
    }
}
```

How can we deal with poor exceptions that come from an external API ??

We can improve our code considerably by **wrapping (adapting) the API** that we are calling and making sure that it returns a meaningful exception type

LocalPort class is just a simple **wrapper** that catches and **transforms exceptions** thrown by the ACMEPort class **into a simpler more meaningful for the caller** exception

80

40

Don't Return Null

```
public void registerItem(Item item) {
    if (item != null) {
        ItemRegistry registry = persistentStore.getItemRegistry();
        if (registry != null) {
            Item existing = registry.getItem(item.getID());
            if (existing.getBillingPeriod().hasRetailOwner()) {
                existing.register(item);
            }
        }
    }
}
```

How about this code ??

81

Don't Return Null

```
public void registerItem(Item item) {
    if (item != null) {
        ItemRegistry registry = persistentStore.getItemRegistry();
        if (registry != null) {
            Item existing = registry.getItem(item.getID());
            if (existing.getBillingPeriod().hasRetailOwner()) {
                existing.register(item);
            }
        }
    }
}
```

How about this code ??

When we return null, we are essentially creating work for ourselves and foisting problems upon our callers.

All it takes is one **missing null check** to send an application spinning out of control.

82

41

Don't Return Null

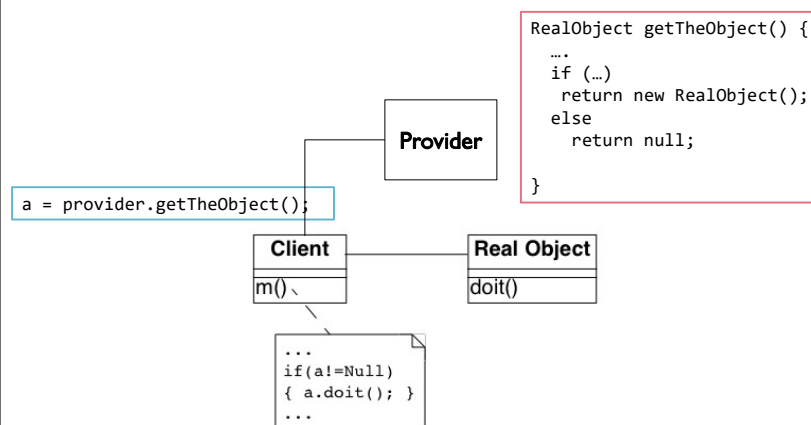
If you are **tempted to return null** from a method, consider

- (1) **throwing an exception**
- (2) **or returning a SPECIAL CASE object instead.**

If you are calling a **null-returning method from a third-party API**, consider **wrapping that method** with a method that either throws an exception or returns a special case object.

83

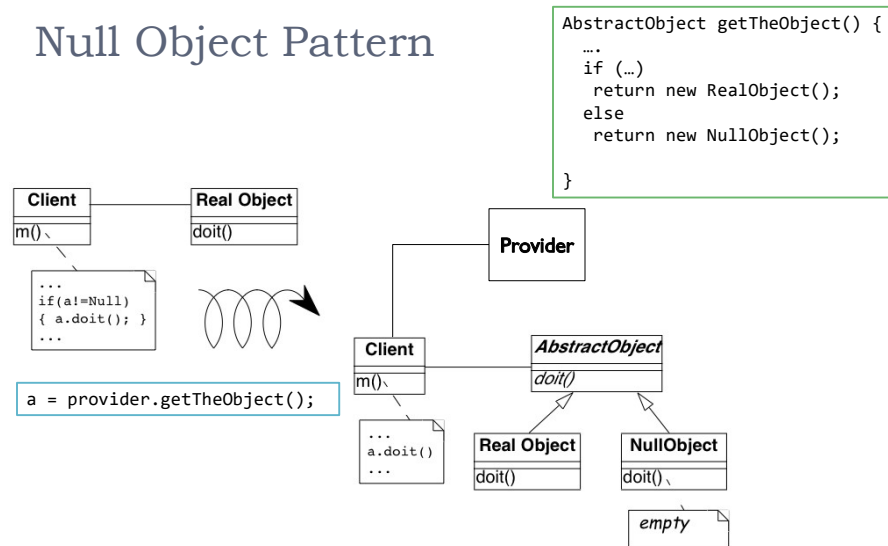
Null Object Pattern



84

42

Null Object Pattern



85

Don't Return Null

```

List<Employee> employees = getEmployees();
if (employees != null) {
    for(Employee e : employees) {
        totalPay += e.getPay();
    }
}

```

Right now, `getEmployees()` can return null, but **does it have to?**

86

43

Don't Return Null

If we change `getEmployees()` so that it **returns an empty list**, we can **clean up the code**. Java has a special method for making immutable empty lists... `Collections.emptyList()`

Or just **return new ArrayList<Employee>()** ;

```
public List<Employee> getEmployees() {
    if( .. there are no employees .. )
        return Collections.emptyList();
}
```

```
List<Employee> employees = getEmployees();
for(Employee e : employees) {
    totalPay += e.getPay();
}
```

87

Don't Pass Null

```
public class MetricsCalculator
{
    public double xProjection(Point p1, Point p2) {
        return (p2.x - p1.x) * 1.5;
    }
    ...
}
```

What happens when someone passes null as an argument?

88

Don't Pass Null

```
public class MetricsCalculator
{
    public double xProjection(Point p1, Point p2) {
        return (p2.x - p1.x) * 1.5;
    }
    ...
}
```

What happens when someone passes null as an argument?

we will get a NullPointerException

89

Don't Pass Null

```
public class MetricsCalculator
{
    public double xProjection(Point p1, Point p2) {
        return (p2.x - p1.x) * 1.5;
    }
    ...
}
```

Could the developer of the class do something better ?

He could **check the arguments** and throw a more **informative exception...**

```
public double xProjection(Point p1, Point p2) {
    if (p1 == null || p2 == null) {
        throw IllegalArgumentException(
            "Invalid argument for MetricsCalculator.xProjection");
    }
    return (p2.x - p1.x) * 1.5;
}
```

90

Don't Pass Null

```
public class MetricsCalculator
{
    public double xProjection(Point p1, Point p2) {
        return (p2.x - p1.x) * 1.5;
    }
}
// Is this better?
```

It might be a little better than a null pointer exception, but remember, we have to define a handler for `InvalidArgumentException`.

```
public double xProjection(Point p1, Point p2) {
    if (p1 == null || p2 == null) {
        throw ArgumentException(
            "Invalid argument for MetricsCalculator.xProjection");
    }
    return (p2.x - p1.x) * 1.5;
}
```

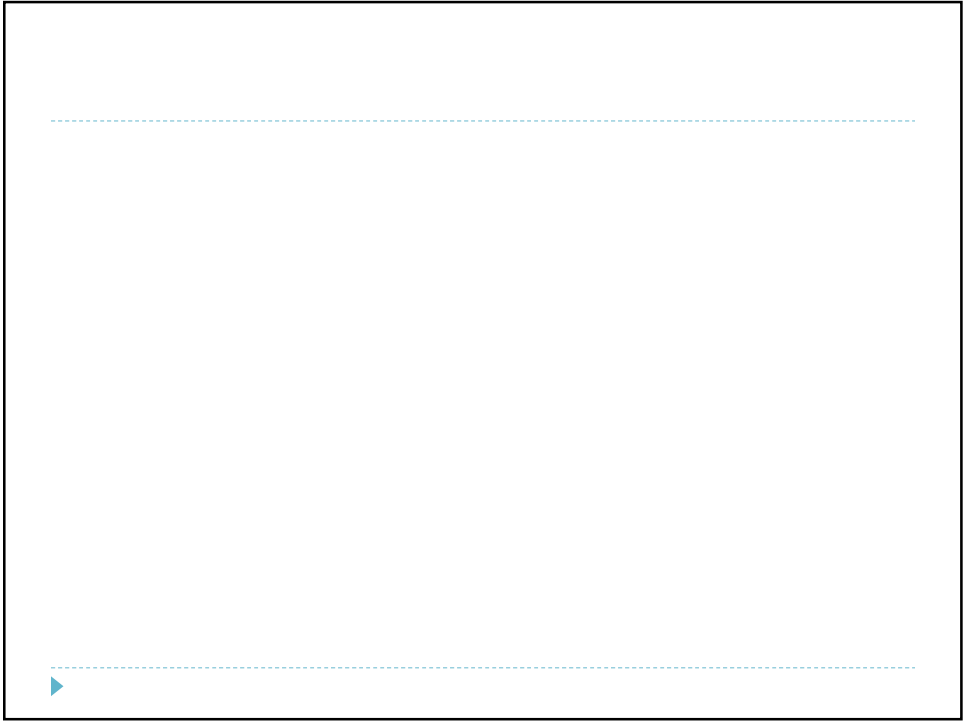
91

Don't Pass Null

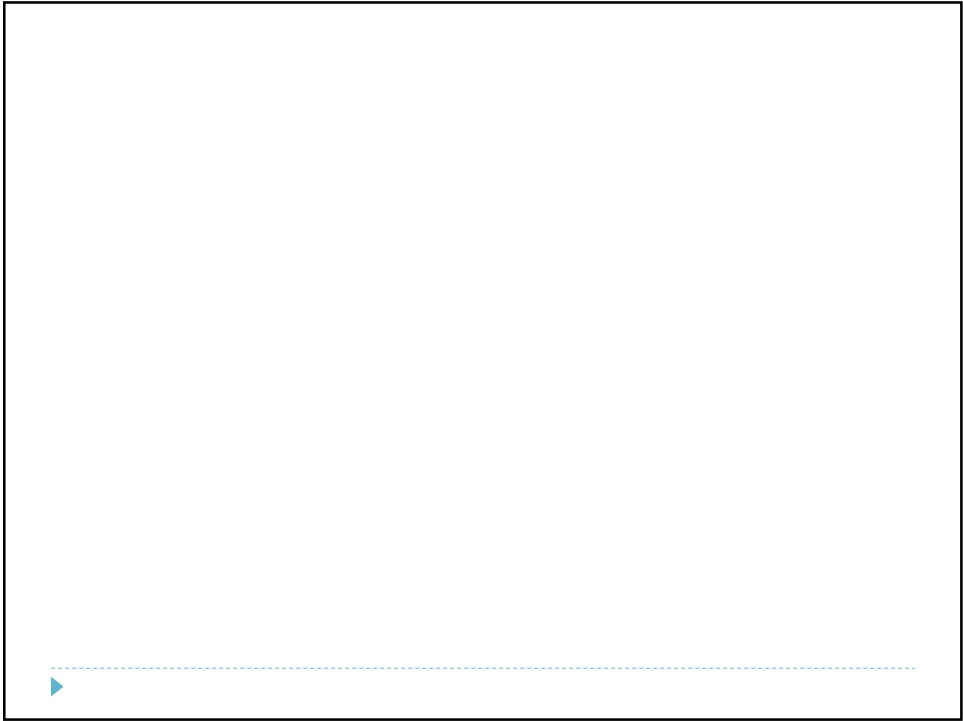
Passing null can only create problems... The caller should check what happens in this case. The called must perform null checks. **Everybody's code becomes more complex.**

Because this is the case, the rational approach is to avoid passing null by default.

92



93



94